

REDES (RCCD) — MACHETE · Guías 1–4 + teóricas + prácticas

1 · TEORÍA DE LA INFORMACIÓN

Info evento $I(s)=-\log_2P(s)$ [bits].
Entropía $H(S)=\sum P(s)\cdot I(s)=-\sum p_i\log_2p_i$ [bits/símbolo]. $0\cdot\log0=0$. $0\leq H\leq\log_2|S|$; $H=0\iff$ algún $p=1$; máx $\iff$ equiprobable $\rightarrow H=\log_2N$. Símbolos independientes: H total = suma (ej. N píxeles de k niveles: $H=N\cdot\log_2k$).

Binaria $H(p_0)=-p_0\log p_0-(1-p_0)\log(1-p_0)$; simétrica, cóncava, 0 en 0 y 1, máx 1 bit en $p_0=1/2$.
Largo prom. $L(C)=\sum P(s)\cdot I(C(s))$. Sin **pérdida** $\iff H(S)\leq L(C)$. Óptimo: $H\leq L<H+1$.
Eficiencia código $\eta=H/L$.

Tipos: instantáneo $\iff$ libre de prefijos (ninguna palabra prefijo de otra) $\Rightarrow$ **univ.**
decodificable (UD: ninguna tira con 2 lecturas; recíproca falsa). No singular: palabras distintas. **Óptimo**: ningún otro código sobre S tiene L menor. Código **sufijo** (ninguna es sufijo de otra) $\Rightarrow$ UD (se decodifica der $\rightarrow$ izq). No UD $\Rightarrow$ hay pérdida de info.

Kraft: $\exists$ código instantáneo con largos $l_i\iff \sum r^{-l_i}\leq 1$ (McMillan: igual para UD). Largo fijo: $L=|\log_2N|$.
Huffman: ordenar por prob; unir los 2 menores en nodo suma; repetir; ramas 0/1; leer raíz $\rightarrow$ hoja. **N equiprobables**: $N=2^k\Rightarrow L=H=k$. Si no: $k=|\log_2N|$, $m=N-2^k\Rightarrow$ 2m símbolos de largo $k+1$ y 2^k-m de largo k ; **$L=k+2m/N$** . $N=6$: $H=2.585$, $L=2.667$; $N=10$: $H=3.322$, $L=3.4$.

Unión $D=A\cup B$ (A equiprob n símb., B m símb.): $P(a)=P_D(A)/n$, $P(b)=P_D(B)/m$; D equiprob $\iff P_D(A)/n=P_D(B)/m$.
 $(1/2,1/2,8,16)No\cdot(1/4,3/4,16,16)No\cdot(1/4,3/4,8,16)No\cdot(3/8,1/8,32,16)Sí(1/48)$.

Check P1E1 $S=(4,3,..,1)$ $H=1.8464$.
 $C1=\{001,01,11,010\}$: 01 prefijo de 010, "01001" $\Rightarrow$ DB o BA $\rightarrow$ no UD ($L=2.5$). $C2=\{0,01,011,111\}$: no inst. pero sufijo $\rightarrow$ UD, $L=1.9$, $\eta=97.2\%$. $C3=\{1,01,001,0001\}$: inst., $L=2$, $\eta=92.3\%$.
P1E2 640×480 px, 10 niveles, 30 img/s, 30dB: $H=307200\cdot\log_210=1\,020\,496.3$ b/img; $Rb=L\cdot30\approx30.61$ Mbps; $B_{min}=Rb/\log_2(1001)\approx3.07$ MHz.

2 · NIVEL FÍSICO

Shannon (ruido):
 $C[bps]=B[Hz]\cdot\log_2(1+SNR)$, SNR lineal: **$SNR=10^{dB/10}$** ; dB=10log₁₀(P_s/P_n). 3dB $\approx\times2$, 10dB=10, 20=100, 30=1000, 100dB=10¹⁰.
 $V_{tx}\leq C$, usar $V_{tx}^{max}=C$ salvo que digan otra. B_{min} para Rb : $B=Rb/\log_2(1+SNR)$. $\log_2x=\ln x/\ln2=\log_{10}x\cdot3.3219$.
Nyquist (sin ruido): $C=2B\cdot\log_2M$ (M niveles). Baudio=símbolo/s;
 $V_i[bps]=V_n[baud]\cdot\log_2M$. bits/símb: BPSK1 QPSK(=4QAM) 2 8PSK3 16QAM4 64QAM6 256QAM8. **Muestreo**: $f_s>2f_{max}$ (voz 4kHz $\rightarrow$ 8000 m/s; CD 44.1k).
PCM: 8000 m/s×8b=64kbps, 1 muestra/c/125µs. **T1**: 24 can×8+1 framing=193b/125µs=1.544Mbps (7b datos $\rightarrow$ 56kbps útil). **E1**: 32 can×8=256b/125µs=2.048Mbps; multitrama 16 tramas=2ms.
Línea: NRZ 0=bajo 1=alto (pierde sincro en rachas). NRZI: 1=transición al inicio, 0=nada (no resuelve 0s). Manchester: transición a mitad de bit ($\uparrow=1$, $\downarrow=0$), **baud=2·bitrate**. Manch. diferencial: 0=transición al inicio, 1=no; siempre a mitad.

SNR mínima: $C\geq Rb\Rightarrow SNR_{min}=2^{Rb/B-1}$. Ej: 640×480, 256 colores equiprob (8b/px), 20 img/s, B=4MHz: $Rb=640\cdot480\cdot8\cdot20=49.152$ Mbps; $Rb/B=12.288\Rightarrow SNR\geq2^{12.288-1}\approx5000\approx37dB$. ($2^x=e^{0.693x}$)

3 · DELAY / CAPACIDAD DE VOLUMEN

$T_{tx}(n)=n/V_{tx}$ ($T_{tx}(1)=1/V_{tx}$). **$T_{prop}=D/V_{prop}$** .
Delay(1)= $T_{tx}(1)+T_{prop}\approx T_{prop}$ (latencia).

Delay(Frame)= $T_{tx}(F)+T_{prop}$.
RTT=2·Delay. **$C_{vol}[bits]=Delay\cdot V_{tx}$** (producto delay×BW; bits "en el caño").
Delay total real: $T_{prop}+T_{tx}+T_{cola}+T_{proc}$.
Redes switcheadas (G2):
 $D=T_{tx}+T_{prop}+T_{queue}$ (store&forward: T_{tx} se paga en cada salto: $\sum(L/R_i+d/v)$).
Convención: el "delay" que da el enunciado = T_{prop} (salvo que diga lo contrario). Mb=10⁶, kb=10³.

Pedido→respuesta:
 $T=T_{tx}(ped)+T_{prop}+T_{tx}(resp)+T_{prop}$.
Checks Luna 385 000km, 100Mbps: $T_{prop}=1.2833s$, RTT=2.567s, $C_{vol}=128.33$ Mbit; pedido 2kb+imagen 25Mb = 2.8167s. $C_{vol}(D,v,SNR,B)$: (100km,200k,100dB,400Hz) $\rightarrow C=13\,287.7$, $C_{vol}=6.64b$. (100km,200k,10dB,400kHz) $\rightarrow C=1\,383\,772.8$, 691.9b · (idem 300k) $\rightarrow461.3b$ · (100m) $\rightarrow0.461b$.

4 · ENLACE PUNTO A PUNTO

Framing: (a) largo fijo $\eta=1$ (sólo framing); (b) campo largo 16b: $\eta=(L-16)/L$; (c) delimitador 8b + bit-stuffing: $\eta=(L-8-n)/L$, n=bits de stuffing (tras cinco 1s se inserta un 0; flag 01111111).
Eficiencia frame $\eta_{frame}=|datos|/|frame|$.
Errores: detectar e errores: $d\geq e+1$; corregir: $d\geq 2e+1$ (d=dist. Hamming mín.).
ARQ = ACK + timeouts. **Stop&Wait**: SWS=RWS=1; secuenciar 2 frames $\rightarrow$ 1 bit SEQ (reencarnaciones: ACK perdido/timeout prematuro $\rightarrow$ duplicado).
Sliding Window: mantener el caño lleno.

$SWS_{opt}=V_{tx}\cdot RTT(F)/|F|$ ($RTT(F)=2\cdot Delay(F)=2(T_{tx}(F)+T_{prop})$)
RWS=SWS si SACK; 1 si no (GoBackN)
#frames $\geq SWS+RWS\rightarrow$
#SEQ= $|\log_2(SWS+RWS)|$. Con k bits y SACK: SWS=RWS=2^{k-1}; GBN: SWS=2^{k-1}
 $\eta_{proto}=T_{tx}(V)/RTT(F)$, $T_{tx}(V)=SWS\cdot|F|/V_{tx}$ (tope 1). S&W: $\eta=T_{tx}(F)/RTT(F)$
|Datos|=|F|-#SEQ-[CRC](-#ACK si piggyback); #frames= $|X|/|Datos|$
 T_{total} (ventana ópt., sin errores)=#frames·|F|/V_{tx}+ T_{prop}
 T_{total} S&W $\approx$ #frames·RTT(F)

GoBackN: RWS=1, ACK acumulativo; error en $F_k\rightarrow$ receptor descarta posteriores, emisor reenvía desde F_k . **SACK/Selective Repeat**: RWS=SWS, receptor bufferea fuera de orden, se reenvía sólo el perdido (NAK); sin buffers SACK no aporta.
Piggybacking: ACK viaja en frames de datos del otro sentido.
Frames modelo — S&W: E: [#SEQ(1)|Datos|Chk] R:[#ACK(1)|Chk]. GBN+piggy: [#SEQ|#ACK|Datos|Chk] ambos. SACK: E:[#SEQ|Datos|Chk] R:[#ACK|#SACK|Chk] (ACK acumulativo + individual). Largo fijo $\rightarrow$ igualar con Padding: [#ACK16|#SACK16|Pad|Chk16].
Receta ventana: 1) $T_{tx}(F)$, T_{prop} , $RTT(F)=2(T_{tx}+T_{prop})$. 2) $SWS=[V_{tx}\cdot RTT/|F|]$ (o la dada). 3) RWS según protocolo. 4) #SEQ= $|\log_2(SWS+RWS)|$. 5) |Datos|, #frames. 6) η o T_{total} : Si SWS < óptima: $T_{total}\approx[\#frames/SWS]\cdot RTT(F)$, $\eta=SWS\cdot T_{tx}(F)/RTT$. Para $\eta=100\%$ con SWS fija: $RTT=SWS\cdot|F|/V_{tx}$ $\rightarrow$ despejar Delay. Con k bits SEQ fijos: máx SWS posible $\rightarrow$ ver si alcanza la óptima.

Diagrama con errores (ej. llegan 4 y 6 con CRC mal): GBN: receptor descarta 4,5,6,7..., ACK acumulativo 3 (o timeout), emisor reenvía desde 4. SACK: bufferea 5,7; ACK 3 + SACK 5,7 (o NAK 4,6); emisor reenvía sólo 4 y 6; luego ACK 7. #SEQ es módulo 2^k (se reciclan).

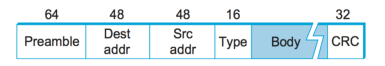
G1E11 S&W 1Mbps, delay .25, F=2kb, CRC16: datos=2000-16-1=1983 $\rightarrow$ 10086 fr; RTT=2(.002+.25)=.504 $\rightarrow$ 5083s $\approx$ 84.7min. 1Gbps: 84.05min (T_{prop} domina). delay .1: RTT=.204 $\rightarrow$ 34.3min.
G1E12 GBN mismo enlace: SWS=1e6. 504/2000=252, RWS=1, #SEQ= $|\log_2253|=8$; datos 1976 $\rightarrow$ 10122 fr $\rightarrow$ T=20.244+.25 $\approx$ 20.5s.
G1E14 F=1kb, 270ms, 1Mbps: RTT=2.271=.542; SWS 7127/255 $\rightarrow$ $\eta=1.29\%/23.4\%/47.0\%$.
E17 SACK SEQ 4b $\rightarrow$ SWS=RWS=8; 10kbps delay 1s F=2kb: $\eta=1.6/2=80\%$.
E19 Marte 60Mbps ACK 1024b [16|16|Pad|16]: SWS=RWS=2¹⁵, frame [SEQ16|Datos992|Chk16] $\eta_{fr}=96.9\%$.

5 · MEDIOS COMPARTIDOS / LAN

MAC: canalización (TDM/FDM/CDMA) o contención. **ALOHA** puro $S=G\cdot e^{-2G}$, máx 18.4% ($G=.5$); ranurado $S=G\cdot e^{-G}$, máx 36.8% ($G=1$). $S=G(1-P_{col})$.
CSMA: sensor; libre $\rightarrow$ transmitir. Ocupado: **1-persistente** (Ethernet) espera y transmite apenas se libera ($\Rightarrow$ 2 esperando colisionan seguro); **no persistente** espera aleatorio y resensa; **p-persistente** transmite con prob p . **CD**: detectar colisión mientras transmite $\rightarrow$ cortar + **jam** $\rightarrow$ **backoff exponencial**: tras k-ésima colisión espera slot $\in[0,2^{k-1}]\times(512\text{ bit-times}=51.2\mu s@10M)$, k tope 10, 16 intentos $\rightarrow$ error.

Peor caso detección = 2 τ (τ =prop. punta a punta máx). **$T_{tx,min}=2\tau\Rightarrow|F|_{min}=V_{tx}\cdot2\tau$** . 10Mbps, $\tau=25.6\mu s\rightarrow51.2\mu s\rightarrow$ **512b=64B**. Una estación que empieza antes de que le llegue la señal ajena ($t<\tau$) sensa libre y colisiona.

Ethernet 802.3 (bits):



Body 46–1500B. Mín sin preámbulo 64B=14+46+4, máx 1518B; MTU 1500. DIX: Type; 802.3: Length (+SOF). Datos <46B $\rightarrow$ **padding**, receptor lo quita con Length. MAC 48b: 3B OUI+3B NIC; broadcast FF:FF:FF:FF:FF:FF. 10base5: 500m/tramo, máx 4 repetidores $\rightarrow$ 2500m.

Dispositivos: repetidor/hub (L1, un dominio de colisión), bridge/switch (L2, 1 dom. colisión por puerto; full-duplex sin CD), router (L3, corta broadcast). LAN = dominio de broadcast.

Learning bridge: aprende **MAC ORIGEN** $\rightarrow$ **puerto de entrada**; destino conocido $\rightarrow$ sólo ese puerto (mismo puerto que origen $\rightarrow$ descarta); desconocido o broadcast $\rightarrow$ **flood** (todos menos el de entrada). Bridge al que no llega la trama no aprende. "¿H ve la trama?" sí si hubo flood hacia su lado o está en el camino. Con ciclos $\rightarrow$ tormentas $\Rightarrow$ STP.

STP BPDU=(switchID, rootID, dist ancia). **Mejor b1<b2**: menor rootID; = $\rightarrow$ menor distancia; = $\rightarrow$ menor switchID (luego menor puerto).

- Root = menor ID; **todos sus puertos DP**.
- Dist. de cada switch = costo mínimo al root (1/segmento).
- Root Port** (1 por switch no-root): donde se ve el mejor BPDU al root (menor dist; empate menor ID del vecino).
- Designated Port** (1 por LAN/segmento): el switch que ofrece mejor BPDU en ese segmento.
- Blocked/Closed**: el resto (no reenvía datos; sí escucha BPDU). Verificar árbol.

Falla de enlace $\rightarrow$ recalcular: puede desbloquear un puerto; RP se mueve. Forwarding sólo por RP/DP. Para **forzar un camino**, elegir IDs: root cerca de lo que querés, el switch que debe bloquear con ID mayor que su vecino del segmento.

G2E1 H1–H4 2500m, 4 hubs, delay máx 25.6µs, 10Mbps: $t_{min}=2\tau=51.2\mu s$; $|F|_{min}=512b=64B$; datos cortos $\rightarrow$ padding (Length). H2,H3 con datos en $t_0+5/+7ms$ mientras H1 transmite 10ms: esperan (1-persist) y **colisionan entre sí** al liberarse. H4 con datos en $t_0+2\mu s$: la señal de H1 aún no llegó $\rightarrow$ sensa libre $\rightarrow$ **colisiona con H1**.
G2E2 X–B1–B2–B3 (Y en B2; Z,W en B3): X $\rightarrow$ W flood, todos aprenden X, Y la ve; Z $\rightarrow$ X directo (B3,B2,B1 aprenden Z; Y no la ve); Y $\rightarrow$ X (B2,B1 aprenden Y; Z no); W $\rightarrow$ Y: B3 flood (Z la ve), B2 directo a Y; B1 no aprende W.
G2E3 L1{S1,S2} L2{S1,S3} L3{S2,S3} L4{S3}: root S1; S2 RP $\rightarrow$ L1, DP $\rightarrow$ L3; S3 RP $\rightarrow$ L2, **bloquea** $\rightarrow$ L3, DP $\rightarrow$ L4. Se corta S2–L1: S2 RP $\rightarrow$ L3 (dist 2), S3 $\rightarrow$ L3 pasa a DP, sin bloqueados.

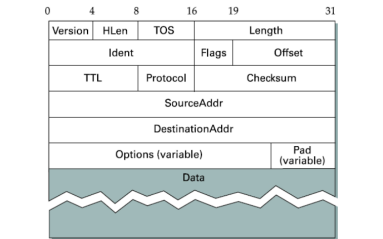
Delay A $\rightarrow$ B con STP: el camino es el del árbol (no el más corto del grafo): segmentos del árbol $\times (T_{tx}+T_{prop})$ (+ T_{queue}). Minimizar: elegir IDs para que el camino A–B quede directo, agregar enlace, subir velocidad, switch en vez de hub. **Frames con MAC origen ajena** en un enlace: sí; los switches no cambian la MAC origen (sólo un router la reescribe) y por flooding cualquier trama aparece en cualquier segmento del árbol.

6 · NIVEL DE RED / IP

	Circuito virtual	Datagrama
Setup	sí (1 RTT antes)	no
Header	VCI corto (local por enlace, se reescribe)	dir. completa orig+dest
Tabla	(p_{in},VCI_{in}) $\rightarrow$ (p_{out},VCI_{out}) 1 fila/conexión	red dest $\rightarrow$ next hop (sin estado)
Falla	caen todos los CV que pasan; re-establecer	sólo paquetes en tránsito; reruteo
Orden/QoS	mismo camino, en orden; reserva recursos fácil	caminos distintos, desorden; QoS difícil

Contar conexiones CV: encadenar out de un switch = in del siguiente.

IPv4 header (20B mín, 60 máx):



Versión va primero porque define el formato. HLen en palabras de 32b (5–15). Length en bytes incl. header, 16b $\rightarrow$ máx **65535B** (no se usa: MTU Eth 1500 $\Rightarrow$ fragmentaria). Router modifica: **TTL–1**, **checksum** (y al fragmentar: Length, MF, Offset; options). Proto: 1 ICMP, 2 IGMP, 6 TCP, 17 UDP, 89 OSPF.

Fragmentación (router si $MTU_{out}<$ datagrama; reensambla sólo el destino; se pierde 1 fragmento $\rightarrow$ se pierde todo): datos/frag= $\lfloor(MTU-20)/8\rfloor\cdot8$; offset en **unidades de 8B**; MF=1 salvo el último (si el original tenía MF=1, el último lo hereda; offsets se suman al original); misma Ident; Length=datos+20. DF=1 y no entra $\rightarrow$ descarta + ICMP 3/4. Ej: 1400B datos por MTU 532 $\rightarrow$ 512/512/376, offsets 0/64/128, MF 1/1/0.

Clases: A 0... 0–127 /8; B 10... 128–191 /16; C 110... 192–223 /24; D 1110 224–239 multicast; E 240–255. **Privadas RFC1918**: 10/8, 172.16/12 (172.16–172.31), 192.168/16 (**168.192.x NO es privada**). 127/8 loopback (no se rutea); 0.0.0.0 este host/ 0/0 default; 255.255.255.255 broadcast limitado; 169.254/16 link-local

(falló DHCP); host todo 1 = broadcast dirigido.

Subnetting /n: h=32-n; direcciones 2^h, **hosts=2^h-2** (red + broadcast). Red=IP AND máscara; broadcast=bits host en 1; hosts red+1...bcast-1. Octeto de máscara: 1b→128,2→192,3→224,4→240,5→248,6→252,7→254,8→255. Bloque=256-valor; red=[oct/bloque]·bloque. /22 255.255.252.0 1022 · /23 .254.0 510 · /24 .255.0 254 · /25 .128 126 · /26 .192 62 · /27 .224 30 · /28 .240 14 · /29 .248 6 · /30 .252 2 (pto a pto). Ej 135.46.56.0/22 → bcast 135.46.59.255 · 147.156.249.228/22 → red .248.0, bcast .251.255 · 10.4.2.0/27 → hosts .1-30, bcast .31.

Tabla forwarding: Red destino → next hop: **interfaz** si directamente conectada; **IP del próximo router** si remota (esa IP debe estar en una red directa); Default 0.0.0.0/0. Una fila por red.

Algoritmo: host: ¿dest en mi red (IP AND máscara)? → directo (ARP al dest) : default gateway (ARP al GW). Router: **longest prefix match** (de las que matchean gana la más específica) → si ninguna → default → si no hay → **descarta** + ICMP dest unreachable (router NO floodea). Next hop IP → se vuelve a buscar para saber interfaz. **Primero ruteo, después ARP.** IP dest no cambia salto a salto; MAC sí.

Errores típicos: máscara mal en interfaz/tabla (host fuera de la subred) · **defaults mutuas R1↔R2 ⇒ loop hasta TTL=0** · ida OK y vuelta perdida: analizar request y reply por separado · gateway fuera de la subred del host.

ARP IP→MAC: request broadcast "¿quién tiene X?", reply unicast, caché con TTL; no cruza routers. En ARP de router, Age "-" = interfaz propia. Gratuitous ARP detecta IP duplicada. **ICMP:** 0 echo reply, 8 echo request (ping), 3 unreachable (code 4 frag needed), 11 time exceeded (traceroute: TTL=1,2,3...), 5 redirect. **DHCP** (UDP 67/68) DORA: Discover/Offer/Request/Ack, 0.0.0.0→255.255.255.255. **NAT:** priv:port↔pub:port.

G3E1 (CV) S1{(1,2)→(3,1),(1,1)→(2,3),(2,1)→(3,2)} S2{(1,1)→(3,3),(1,2)→(3,2)} S3{(1,3)→(2,1),(1,2)→(3,1)} ⇒ 3 conexiones A→D, A→B, B→E (unidireccionales). Cae S2 → caen A→D y B→E. **G3E4** A: 56.0/22→If1, 60.0/22→If0, 192.53.40.0/23→.60.50, /24→.60.100, Def→.62.100. B (sin default): 56.0/25, 60.0/25, 192.53.40/23. 192.53.40.7 en A matchea /23 y /24 → gana /24 → 135.46.60.100 (€60.0/22 → sale If0). B descarta 57.14 (fuera del /25 .0-127). **G3E6** browser 192.168.2.90/25, R2 IF 192.168.2.62/26 y rutas /26 (.0-63) ⇒ request llega, el **reply** cae en default mutua R1↔R2 → loop hasta TTL=0. Fix: los 3 /26 → /25. **G3E14** ARP Age "-" = IF propia; tabla MAC del switch → qué equipo en qué puerto; 127.0.0.1 no se rutea; 192.168.1.1 (privada) igual va por default; 168.254.255.255 = broadcast dirigido del /16 → R2.

Asignar IPs: 1 subred por LAN y por enlace router-router (/30); router .1, hosts siguientes; PCs: red directa + default a su router; host en red con 2 routers: rutas específicas a cada lado (o default + 1 específica). Router: redes directas → IF, remotas → IP del vecino en la subred compartida.

7 · RUTEO
Forwarding=consultar tabla y elegir salida; **Routing**=construir tablas (grafo, camino mínimo). Estático (manual, cero overhead, no reacciona) vs dinámico (protocolo, se adapta, overhead). **IGP** (intra-AS):

DV→**RIP**, LS→**OSPF**. **EGP** (inter-AS): path vector→**BGP**.

Métricas: RIP=1 por salto; **OSPF=10¹⁰/BW[bps]** (10M→1000, 100M→100, 1G→10).

Frames de la cátedra: RIP [RED|COSTO] · OSPF [ID|NUM SEQ|TTL|RED|COSTO].

Distance Vector / RIP
Cada nodo manda **su vector completo (destino, costo)** sólo a **vecinos directos**, periódico (RIP 30s) + triggered. Bellman-Ford: **D_x(y)=min_v{c(x,v)+D_v(y)}**; acepta si es mejor, o si viene de su next hop actual (aunque empeore). La tabla vieja no cuenta, sólo vectores de vecinos + costo al vecino.

Ejercicio matriz global: fila=nodo, diag 0; boot: 1 a vecinos, ∞ resto; cada ronda: fila=min sobre vecinos(fila vecino+1); tras k rondas conoce hasta k+1 saltos; converge en ~diámetro. Tabla: Destino|Costo|Próx. salto (∞, "-").

Mensaje RIP en IP: red conectada costo 0 (convención práctica) o 1; aprendida = recibido+1. Deducir topología: costo 0 ⇒ el emisor está conectado; costo k ⇒ k routers más allá.

Count-to-infinity: cae enlace, vecino anuncia ruta vieja que pasaba por mí → suben de a poco ("bad news travel slow", good news fast). Parches: ∞=16 (RIP máx 15 saltos), **split horizon** (no anunciar al vecino lo aprendido de él), **poison reverse** (anunciárselo con ∞). **Sólo resuelven lazos de 2 nodos** (triángulo sigue fallando).

Link State / OSPF

Cada nodo manda **sólo sus enlaces directos** (vecino, costo) a **toda la red por flooding confiable**; todos arman LSDB idéntica y corren **Dijkstra**. Pasos: descubrir vecinos (HELLO) → medir costo → armar LSP → flood → SPF. LSP=<ID, SEQ, TTL/age, lista vecinos>. Flooding: guardo el LSP más reciente por nodo; si SEQ ≤ guardado → descarto (no reenvío); si nuevo → ACK y reenvío a todos menos al que me lo mandó; TTL decrementa (llega con TTL-1 por cada router intermedio), TTL=0 → borrar; reinicio → SEQ=0. **SEQ** descarta viejos/duplicados; **TTL/age** evita LSP eternos. Contradicción de LSPs: mirar SEQ/age, no hora de llegada; enlace vale si lo confirman ambos extremos (two-way). Mensaje: sólo costos directos, el contenido no cambia entre iteraciones (sólo SEQ/TTL).

Forward search: Confirmed={ (yo,0,-) }; tomar Next=último confirmado, por cada vecino N: costo=C(Next)+l(Next,N); si no está en ninguna lista → Tentative (NextHop heredado de Next, o N si Next=yo); si está en Tentative con más costo → reemplazar. Mover el menor de Tentative a Confirmed; repetir hasta vacío.

Check A-B5,A-C2,B-C2,B-D2,C-D1: A recibe B:{A5,C2,D2}, C:{A2,B2,D1}, D: {B2,C1} (TTL X-1). Dijkstra: C2 → (B4 vía C, D3 vía C) → D3 → B4. Todo por C. A-C3,A-D8,C-E1,C-F6,B-E2,D-E2: A→D por C (3+1+2=6 < 8).

	Link State (OSPF)	Distance Vector (RIP)
Manda	sus enlaces	su vector entero
A quién	a todos (flood)	sólo vecinos
Cuándo	on-event + refresh	periódico 30s + triggered
Memoria	topología entera O(E)	vector O(N)
CPU	Dijkstra O(E log N)	min local (barato)
Converg.	rápida, sin count-to-∞	lenta, count-to-∞
Escala	mejor (áreas)	chica (≤15 saltos)

¿Camino más corto ∀t? Estático: sólo si la tabla está al día. OSPF: si convergió. RIP: sólo si min saltos = min costo y está al

día. **Flooding: siempre** (encuentra todos los caminos, ineficiente).

Overhead (bps)=bits por ciclo / período. (G3E13: header 32b, dir 32b, métrica 32b, 30s)
RIP: msg=32+64·R (R destinos); msg/ciclo=2·#enlaces (cada router a cada vecino) → bits=2E·(32+64R). **OSPF:** LSA=32+64·grado; cada LSA cruza cada enlace → bits=E·ΣLSA_i (flooding real con ciclos: más, por duplicados). Σgrados=2E.
5 nodos: línea/estrella (E=4): RIP 2816b→93.9bps, OSPF 2688b→89.6bps. K5 (E=10): RIP 7040→234.7, OSPF 14400→480bps.

RIP práct. Ej3 R1 (10.0.2.1/24, 10.0.3.1/30) recibe: 10.0.2.0/24 1 · 10.0.3.0/30 0 · 10.0.4.0/24 0 · 10.0.5.0/24 1 ⇒ vecino R2 conectado a .3/30 y .4/24, R3 detrás con .5. R1 manda: 2/24 0, .3/30 0, .4/24 1, .5/24 2. Tabla R1: .2→IF0/0, .3→IF0/1, 4 y .5→10.0.3.2. **OSPF práct. Ej2** msg de R1 = [ID R1|TTL X|#Seq W] + (red,costo) de sus redes directas: 161.139.21.64/26 10 (1G), 192.168.5.0/30 10, 192.168.1.0/30 1000 (10M). Tabla: evita el 10M yendo por el 1G. **G3E8 matriz DV** (A-B,A-C,A-E,A-F,B-C,C-D,D-G,F-G): ronda 1 msg A: A0 B1 C1 E1 F1; ronda 2: +D2 (por C), G2 (por F); converge al llenar pares a 3 saltos (B↔G, D↔E, E↔G).

8 · TRANSPORTE (UDP/TCP)

Puerto: id 16b de un proceso/endpoint dentro del host. **Multiplexación** (emisor agrega header con puertos) / **demux** (receptor entrega por puerto; TCP por 4-tupla IPo:Po↔iPd:Pd). Sin puertos: una sola app de red por host. Server escucha puerto fijo; cliente usa efímero.

	TCP	UDP
Conexión	sí (3-way / 4-way)	no
Confiable	ACK, checksum, reTx por timeout, orden	best effort
Flujo/Con g.	sí / sí	no / no
Datos	stream bytes, full-duplex	datagramas
Uso	web, mail, archivos	DNS, VoIP/RTP, juegos

UDP header 8B: SrcPort16|DstPort16|Length16|Checksum16.

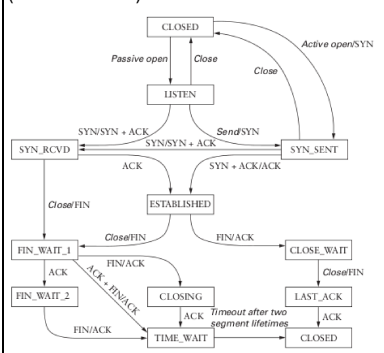
TCP header (20B min, 60 máx): SrcPort16|DstPort16|SeqNum32|Ack32|HdrLen4|Res6|Flags6(URG ACK PSH RST SYN FIN)|AdvWindow16|Checksum16 (pseudo-header IP+TCP+datos)|UrgPtr16|Options. MSS def 536 (MTU 576-40); Eth: 1460.

Seq=nº del 1er byte del segmento; **Ack**=próximo byte esperado (acumulativo). **SYN y FIN consumen 1 nº de seq:** ACK puro no. seq_{sig}=seq+len(+1 si SYN/FIN). ISN aleatorio. ack repetido = no llegó nada nuevo del otro.

Enlace vs transporte (G4E2): comparten SEQ, ACK, checksum, ARQ/timeout, ventana. Transporte agrega: setup/release explícito, **timeouts adaptativos** (RTT variable), paquetes muy viejos (espacio SEQ grande, TIME_WAIT), control de flujo con ventana dinámica, control de congestión. Red CV sin control de errores: no se puede multicamino; full-duplex sí (2 simplex); SEQ sí (detectar pérdidas, no para ordenar).

3-way: C→S SYN seq=J · S→C SYN+ACK seq=K ack=J+1 · C→S ACK ack=K+1. **Cierre:** FIN seq=M → ack M+1 → FIN seq=N → ack N+1 (ACK+FIN pueden ir juntos → 3 segmentos). Half-close: el otro sigue mandando datos tras recibir FIN.

Máquina de estados TCP (evento/acción)



Cliente ESTAB al **mandar** el 3er ACK; server al **recibirlo**. Va a TIME_WAIT quien cerró primero (manda el último ACK). netstat CLOSE_WAIT = el otro cerró y mi app no; SYN_SENT = sin respuesta al SYN (firewall/host caído).

Monitor sin estado: el SYN+ACK es el mejor paquete (único con ambos flags, lo manda el server ⇒ roles; prueba aceptación). SYN solo puede ser scan; ACK indistinguible. **RST:** resetea conexión confusa (puerto cerrado responde RST; segmento incoherente/SYN en conexión establecida/ack fuera de ventana → RST; inyección/hijack off-path). SYN scan (nmap): SYN→SYN+ACK→RST (nunca completa).

Flujo: AdvWin=MaxRcvBuf-(LastByteRcvd-LastByteRead); EffWin=AdvWin-(LastByteSent-LastByteAcked); Win=0 → **persist timer** (probes de 1B). Throughput≤Win/RTT; caño lleno si Win=RTT·BW (AdvWin 16b → 64KB máx; RFC1323 window scale). **Nagle:** si hay datos sin ACK en vuelo, bufferear hasta ACK o MSS.

RTT/Timeout. Original: EstRTT=α·EstRTT+(1-α)·Sample, α∈[.8,.9]; TO=2·EstRTT. **Karn/Partridge: no muestrear segmentos retransmitidos** (ambigüedad del ACK); duplicar TO en cada reTx. **Jacobson/Karels:** Diff=Sample-EstRTT; EstRTT+=δ·Diff; Dev+=δ(|Diff|-Dev) (δ≈1/8); **TO=μ·EstRTT+φ·Dev, μ=1, φ=4.** Muestras válidas: SYN↔SYN+ACK, SYN+ACK↔ACK, datos↔su ACK, FIN↔ACK.

G4E5 2 conexiones por 4-tupla; CONN1 cierre FIN / FIN+ACK / ACK (server active close → TIME_WAIT, cliente CLOSE_WAIT→LAST_ACK→CLOSED); CONN2: SAU con ack=64334 fuera de ventana → RA (reset) ⇒ inyección / segmento viejo. **G4E7** 6/7 = mismo segmento reTx (seq 302 len 1000) → no sirve para RTT; 4-way sin comprimir: server FIN_WAIT_1→FIN_WAIT_2→TIME_WAIT; cliente CLOSE_WAIT→LAST_ACK→CLOSED.

Leer trazas: agrupar por 4-tupla (sin importar sentido); cliente = quien manda el 1er SYN (puerto efímero), server = puerto conocido (80 http, 443 https, 22 ssh, 53 dns, 25 smtp, 520 rip, 179 bgp); ack del otro = seq+len (+1 SYN/FIN); mismo seq+len repetido = retransmisión; si el ack no coincide con lo enviado → anómalo. **netstat:** LISTEN 0.0.0.0:p = server escuchando; TIME-WAIT = yo cerré primero; CLOSE-WAIT = el otro cerró; SYN-SENT = esperando SYN+ACK; una conexión entrante a mi puerto en LISTEN crea nueva entrada ESTABLISHED con el mismo puerto local. **RST inyectado** por un tercero (gateway malicioso) con seq en ventana → el receptor aborta, conexión muere sin FIN. **Ventana ej.:** buffer RX 4K; llega 2K → ACK 2048 WIN 2048; +2K → WIN 0 (emisor bloqueado); app lee 2K → WIN 2048. **TIME_WAIT 2·MSL:** que mueran duplicados viejos y poder reenviar el último ACK.